MUJ

MANIPAL UNIVERSITY JAIPUR

(University under Section 2(f) of the UGC Act)

Department of Computer Applications

Course Hand-out – Practical / Laboratory Component

LAB MANUAL – CAP6130 : WEB TECHNOLOGIES LAB

Programme Name	MCA
Course Name	Web Technologies Lab
Course Code	CAP6130
LTPC	0-0-2-1
Session	Jul – Nov 2026
Class	I Semester
Course Coordinator	Dr. Timothy Malche
Course Instructor(s)	Dr. Timothy Malche

Table of Contents

No.	Experiment Title	CO Mapped
1	HTML5 Forms and Input Elements	CAP6130.1
2	CSS Styling, Box Model and Responsive Layout	CAP6130.1
3	HTML Tables – Merging Rows and Columns	CAP6130.1
4	Introduction to XML – Well-Formed Documents	CAP6130.2
5	PHP Basic Syntax – Variables, Operators and Output	CAP6130.3
6	Decision-Making and Looping in PHP	CAP6130.3
7	Integrating PHP with HTML – Dynamic Content Generation	CAP6130.3
8	Arrays and Functions in PHP	CAP6130.3
9	String Handling and Browser Detection in PHP	CAP6130.3
10	Multipage Form Processing and Server-Side Validation	CAP6130.4
11	File Handling, Error Handling and Basic Security	CAP6130.4
12	Cookies and Sessions – Maintaining User State	CAP6130.5
13	Object-Oriented Programming with PHP	CAP6130.5
14	PHP–MySQL Connectivity and Database Management	CAP6130.5
15	Full-Stack CRUD Application – Student Record Management System (Mini Project)	CAP6130.5

Course Outcomes (CO)

On successful completion of this laboratory course, students will be able to:

CO	CO Statement
CAP6130.1	Explain the use of HTML for structuring web content and CSS for styling, layout, and responsive web design.
CAP6130.2	Explain the fundamental concepts, structure, and use of XML.
CAP6130.3	Develop dynamic and interactive web pages using PHP.
CAP6130.4	Analyze web applications for performance and validate forms to ensure data integrity and security.
CAP6130.5	Design and implement full-stack web applications integrating PHP, MySQL, cookies, sessions, and object-oriented programming features.

General Instructions to Students

- Install and configure XAMPP / WAMP (Apache, PHP, MySQL) and a code editor (e.g. VS Code) before the first lab session.
- Come prepared with the algorithm / pseudocode for the given task before entering the lab.
- Save every program using the naming convention RollNo_ExpNo_Title (e.g. 2401_Exp05_BasicSyntax.php).
- Test each program for at least two different sets of input (including invalid input, where applicable) before demonstration.
- Maintain a lab record containing the aim, code, output screenshots and learning outcome for every experiment.
- Be ready to answer viva-voce questions related to the concepts used in the experiment.
- Academic integrity is mandatory – copied programs will not be evaluated.

Experiment 1: HTML5 Forms and Input Elements

Maps to Course Outcome: CAP6130.1

Objective

Design a semantically correct, accessible HTML form using a range of HTML5 input types and grouping elements.

Task

a) Create an HTML page titled “Online Course Enrolment Form” that includes: text input (name), email input, password input, radio buttons (gender), checkboxes (areas of interest), a drop-down list (course selection), a date picker (date of birth), a file upload field (profile picture), a range slider (experience level), and Submit / Reset buttons.

b) Apply HTML5 validation attributes such as required, placeholder, pattern, min, max and autofocus on appropriate fields.

c) Group related fields using <fieldset> and <legend> (e.g. “Personal Details”, “Course Preferences”).

d) View the form in the browser and verify that built-in HTML5 validation (e.g. blocking submission on an invalid email) works without any JavaScript.

Expected Learning Outcome

Students can construct a well-structured, self-validating HTML5 form using modern input types and grouping elements.

Reference: WHATWG HTML Living Standard – Forms

Experiment 2: CSS Styling, Box Model and Responsive Layout

Maps to Course Outcome: CAP6130.1

Objective

Apply CSS to style page elements and build a layout that adapts to different screen sizes.

Task

a) Build an HTML page that displays four “product cards” (product image, name, price and a “Buy Now” button) arranged using CSS Flexbox or Grid.

b) Style each card with rounded corners (border-radius) and a box-shadow, and add a hover effect that enlarges the shadow / scales the card slightly.

c) Create a separate with rounded corners and a yellow background that contains an image and a heading; the heading text and paragraph text inside must use two different colours.

d) Add a CSS media query so that on screens narrower than 600px the cards stack into a single column instead of a row.

e) Use browser Developer Tools to inspect the box model (content, padding, border, margin) of one card and note the computed values.

Expected Learning Outcome

Students understand the CSS box model, visual styling properties, and how to write a basic responsive breakpoint.

Reference: W3C CSS Backgrounds and Borders Module | W3C CSS Box Model Module

Experiment 3: HTML Tables – Merging Rows and Columns

Maps to Course Outcome: CAP6130.1

Objective

Present structured tabular data correctly using semantic table markup and merged cells.

Task

a) Create a weekly class timetable (Monday–Friday, 6 periods per day) as an HTML table using <thead> and <tbody>.

b) Use colspan to merge cells for a 2-period-long laboratory session, and rowspan to merge cells for a subject that repeats across two consecutive periods on the same day.

c) Add a <caption> describing the table and style it with CSS – alternating row colours, a header row background colour, and cell borders.

d) Validate that the merged layout renders correctly and the table remains readable on a smaller viewport (add horizontal scrolling with CSS if needed).

Expected Learning Outcome

Students can merge table cells correctly and structure tables semantically for real-world data presentation.

Reference: WHATWG HTML Living Standard – Tables

Experiment 4: Introduction to XML – Well-Formed Documents

Maps to Course Outcome: CAP6130.2

Objective

Create a well-formed XML document and understand the difference between well-formedness and validity.

Task

a) Design an XML document catalog.xml that represents a “Library Catalog” containing a single root element <library> with at least five <book> child elements.

b) Each <book> element should have child elements <title>, <author>, <year> and <price>, and a genre attribute (e.g. <book genre="fiction">).

c) Ensure the document follows XML well-formedness rules: a single root element, properly nested and closed tags, case-sensitive tag names, and correctly quoted attribute values.

d) Write a simple DTD (catalog.dtd) that declares the structure of the document, link it to the XML file, and validate the document against it.

e) Open the XML file directly in a browser and observe how it is displayed compared to an HTML page; note the purpose of the XML declaration <?xml version="1.0"?>.

Expected Learning Outcome

Students can author syntactically correct XML, understand element/attribute design, and validate a document against a DTD.

Reference: W3C Extensible Markup Language (XML) 1.0

Experiment 5: PHP Basic Syntax – Variables, Operators and Output

Maps to Course Outcome: CAP6130.3

Objective

Write and execute PHP scripts that use core language constructs correctly.

Task

a) Write a PHP script that declares variables of different data types (string, integer, float, boolean) and a constant using define(), then prints all of them using echo, print and print_r().

b) Write a script that accepts two numbers through an HTML form and applies arithmetic, comparison and logical operators on them, displaying every result in a neatly formatted HTML table.

c) Demonstrate the difference between a local variable and a global variable using the global keyword inside a function.

d) Use var_dump() to inspect and explain the type and value of each variable used above.

Expected Learning Outcome

Students become comfortable with PHP syntax fundamentals, data types, operators and output statements.

Reference: PHP Manual – Basic Syntax

Experiment 6: Decision-Making and Looping in PHP

Maps to Course Outcome: CAP6130.3

Objective

Apply conditional and iterative control structures to solve simple programming problems in PHP.

Task

a) Write a PHP program that accepts a number from an HTML form and reports whether it is Positive / Negative / Zero and Even / Odd, using if-elseif-else.

b) Write a PHP program using switch-case that converts a numeric grade (0–100) entered by the user into a letter grade (A/B/C/D/F).

c) Using a for loop, print the multiplication table of a number entered by the user.

d) Using a while or do-while loop, write a program to compute the factorial of a number and to check whether it is a prime number.

Expected Learning Outcome

Students can implement conditional branching and looping constructs to solve computational problems in PHP.

Reference: PHP Manual – Control Structures

Experiment 7: Integrating PHP with HTML – Dynamic Content Generation

Maps to Course Outcome: CAP6130.3

Objective

Embed PHP code within HTML pages to generate dynamic, data-driven content.

Task

a) Create an HTML form where a student enters their name and marks for five subjects. On submission, a PHP page should calculate the total, percentage and grade, and display the result embedded inside a styled HTML page (PHP and HTML combined in the same file).

b) Use PHP to dynamically generate an HTML table that lists each subject and its marks, followed by the computed total and grade in the final row.

c) Demonstrate the difference between the short-echo tag <?= $variable ?> and the full <?php echo $variable; ?> syntax within the same page.

d) Add conditional PHP logic so the result row is shown in green text for a passing grade and red text for a failing grade.

Expected Learning Outcome

Students understand how PHP output merges with HTML markup to build a complete dynamic page.

Reference: PHP Manual – PHP and HTML

Experiment 8: Arrays and Functions in PHP

Maps to Course Outcome: CAP6130.3

Objective

Use indexed and associative arrays together with built-in and user-defined functions to organise PHP code.

Task

a) Create an indexed array of 10 numbers and write a user-defined function that returns the maximum, minimum and average of the array without using any built-in aggregate function.

b) Create an associative array of student-name ⇒ marks pairs; use a foreach loop to display all records and identify the topper.

c) Demonstrate at least five built-in array functions such as sort(), array_push(), in_array(), array_search() and count() on a sample array.

d) Write one function that uses a default argument value and another that accepts a variable number of arguments using the ...$args syntax.

Expected Learning Outcome

Students gain confidence in array manipulation and in designing reusable, well-parameterised PHP functions.

Reference: PHP Manual – Arrays | PHP Manual – Functions

Experiment 9: String Handling and Browser Detection in PHP

Maps to Course Outcome: CAP6130.3

Objective

Manipulate strings using PHP's built-in functions and read client/browser information using PHP superglobals.

Task

a) Write a PHP program that accepts a string from an HTML form and: (a) checks whether it is a palindrome, (b) counts the number of vowels, and (c) reverses the string – using functions such as strlen(), strrev(), substr() and str_replace().

b) Write a PHP script that reads $_SERVER['HTTP_USER_AGENT'] to detect and display the visitor's browser name and operating system.

c) Build a simple contact form where PHP validates, using a regular expression (preg_match), that the Name field contains only alphabets and spaces.

Expected Learning Outcome

Students can process strings confidently and use PHP superglobals to read information about the client request.

Reference: PHP Manual – String Functions | PHP Manual – $_SERVER

Experiment 10: Multipage Form Processing and Server-Side Validation

Maps to Course Outcome: CAP6130.4

Objective

Carry form data across multiple pages and validate it on the server before it is accepted.

Task

a) Build a three-page PHP application: Page 1 collects personal details (name, email, phone), Page 2 collects address details, and Page 3 displays a summary of all the data entered, carrying values forward using $_SESSION (or hidden fields with $_POST).

b) Add server-side validation on Page 1 and Page 2: required-field checks, a valid email format using filter_var($email, FILTER_VALIDATE_EMAIL), and a 10-digit phone number pattern – redisplay the form with clear error messages if validation fails.

c) Sanitise every input using htmlspecialchars() before it is redisplayed, to prevent basic HTML/script injection.

Expected Learning Outcome

Students gain an end-to-end understanding of multi-step form workflows and secure server-side input handling.

Reference: PHP Manual – filter_var()

Experiment 11: File Handling, Error Handling and Basic Security

Maps to Course Outcome: CAP6130.4

Objective

Read and write files from PHP, handle runtime errors gracefully, and recognise a common web security risk.

Task

a) Write a PHP script that appends every submitted feedback-form entry to a text file feedback.txt using fopen(), fwrite() and fclose().

b) Write a script that reads and displays the contents of feedback.txt line-by-line using fgets() or file().

c) Demonstrate try-catch exception handling for two scenarios: attempting to open a file that does not exist, and dividing a number by zero.

d) Examine a given PHP code sample that builds a SQL query by directly concatenating user input, explain why it is vulnerable to SQL Injection, and rewrite it safely using a parameterised (prepared) statement.

Expected Learning Outcome

Students can perform file I/O safely, handle exceptions in a structured way, and recognise / fix a SQL-Injection-prone query.

Reference: PHP Manual – Filesystem Functions | PHP Manual – Exceptions

Experiment 12: Cookies and Sessions – Maintaining User State

Maps to Course Outcome: CAP6130.5

Objective

Implement stateful behaviour for a user across otherwise-stateless HTTP requests.

Task

a) Write a PHP script that sets a cookie storing the user's name and the current date/time when they visit, and on their next visit displays “Welcome back, <name>! Your last visit was on <date>.”

b) Implement a session-based login system: a login page checks a submitted username/password against fixed credentials, and on success calls session_start() and stores the username in $_SESSION.

c) Create a “dashboard” page that checks isset($_SESSION['username']) at the top of the script and redirects back to the login page if the session is not set.

d) Add a Logout link that calls session_unset() and session_destroy(), and verify that the dashboard can no longer be accessed afterwards.

Expected Learning Outcome

Students clearly understand the difference between cookies and sessions and can choose correctly between them.

Reference: PHP Manual – Sessions | PHP Manual – Cookies

Experiment 13: Object-Oriented Programming with PHP

Maps to Course Outcome: CAP6130.5

Objective

Apply core object-oriented principles – classes, objects, inheritance and encapsulation – in PHP.

Task

a) Design a class Student with properties (name, rollNumber, an array of marks) and methods __construct(), addMarks(), calculateAverage() and displayDetails().

b) Create a child class GraduateStudent that extends Student, adds a thesisTitle property, and overrides displayDetails() to also show the thesis title (demonstrating inheritance and method overriding).

c) Make the properties private or protected and provide public getter/setter methods to demonstrate encapsulation.

d) In a single script, create multiple Student and GraduateStudent objects and display all of their details using a loop.

Expected Learning Outcome

Students gain hands-on experience designing classes and applying inheritance and encapsulation in PHP.

Reference: PHP Manual – Object-Oriented PHP

Experiment 14: PHP–MySQL Connectivity and Database Management

Maps to Course Outcome: CAP6130.5

Objective

Connect a PHP application to MySQL and manage databases and tables through both phpMyAdmin and PHP code.

Task

a) Using phpMyAdmin, create a database named college_db and, within it, a table students with columns id (auto-increment primary key), name, course and marks.

b) Write a PHP script using mysqli (or PDO) that connects to college_db and prints a clear success or failure message.

c) Using PHP, programmatically list all databases on the server and all tables inside college_db (using SHOW DATABASES and SHOW TABLES executed through PHP).

d) Write a PHP script that alters the students table to add a new column email, and verify the change in phpMyAdmin.

Expected Learning Outcome

Students can set up a database, connect to it from PHP, and perform basic DDL operations both through a GUI tool and through code.

Reference: PHP Manual – MySQLi Extension | MySQL 8.0 Reference Manual

Experiment 15: Full-Stack CRUD Application – Student Record Management System (Mini Project)

Maps to Course Outcome: CAP6130.5

Objective

Integrate HTML, CSS, PHP, MySQL, sessions and OOP into one complete, working full-stack application.

Task

a) Build a session-protected "Student Record Management System" (only a logged-in user, per Experiment 12, may access it) built around the students table from Experiment 14.

b) Create: provide an HTML form and PHP script that inserts a new student record into the database.

c) Read: build a PHP page that fetches and lists all student records from MySQL in a styled HTML table.

d) Update: build an edit page that is pre-filled with a selected student's existing data and saves changes back to the database.

e) Delete: add a delete action, with a confirmation step, that removes a selected record from the database.

f) Refactor the database operations to go through the Student class from Experiment 13, so the application combines OOP design with MySQL access (rather than writing raw queries directly on each page).

Expected Learning Outcome

Students demonstrate the ability to combine every major concept from the course – forms, PHP, sessions, OOP and MySQL – into one working full-stack CRUD application.

Reference: PHP Manual – MySQLi Extension | PHP Manual – Object-Oriented PHP

Name & Signature
(Lab In-charge)